Fall 2025 CPTS530 Final Project
MATH 448-548
Numerical Analysis

Aryan Ritwajeet Jha

11 December 2025

Contents

Problem 1: Orthogonal Matching Pursuit (OMP)

Problem Statement:

Part 0: Preliminary Information and Data

In part 2, use the matrix

$$A = \begin{bmatrix} 1 & -1/2 & -1/2 \\ 0 & \sqrt{3}/2 & -\sqrt{3}/2 \\ 1 & 3 & 3 \end{bmatrix}$$

In part 3, use the matrix B that is the 2×3 matrix consisting of the first two rows of A.

Singular Value Decomposition (SVD) and Pseudo-Inverse

Given a matrix $M_{m \times n}$ with $\text{rank}(M) = r$, write

$$M = U \Sigma V^T$$

where $U_{m \times m}$ and $V_{n \times n}$ are orthogonal matrices, and $\Sigma_{m \times n}$ is a diagonal matrix with nonzero diagonal entries $\sigma_1 \geq \sigma_2 \geq \ldots \geq \sigma_r > 0$ called singular values of M.

Definition. Given M as above, the pseudo-inverse M^+ is defined by

$$M^+ = V \Sigma^+ U^T$$

where $\Sigma^+_{n \times m}$ is the diagonal matrix whose non-zero entries are $\frac{1}{\sigma_1}, \frac{1}{\sigma_2}, \ldots, \frac{1}{\sigma_r}$.

Thresholding Operator

The **hard thresholding operator** H_s is defined as follows:

$$\mathbf{x} \to H_s(\mathbf{x}) \in \mathbb{R}^n$$

$H_s(\mathbf{x})$ sets all entries of $\mathbf{x}$ to zero except for s largest (in absolute value) entries.

The Matrix $A_{\mathcal{S}}$

Given a matrix A and an index set $\mathcal{S}$, denote by $A_{\mathcal{S}}$ the matrix consisting of columns of A with index in $\mathcal{S}$.

Assignments and Questions

1. Implement the pseudocode for the Orthogonal Matching Pursuit (OMP). The pseudocode is contained in a separate handwritten note attached to the Final project assignment. You may use the standard Matlab (or other language) commands to compute SVD and pseudo-inverse.

2. Run the code for $\mathbf{b} = A\mathbf{e}_1$ with $\mathbf{e}_1 = (1, 0, 0)^T$. See how well the obtained vector $\mathbf{x}$ matches $\mathbf{e}_1$.

3. Run the code again, this time using the matrix B instead of A.

4. Compare the results obtained in parts 2, 3. Explain the connection with compressed sensing and sparse recovery.

Part 1: OMP Implementation

The Orthogonal Matching Pursuit algorithm was implemented following the provided pseudocode. Key components include:

- Hard thresholding operator $H_s(\mathbf{x})$ that retains only the s largest entries
- OMP iteration with normalized correlation-based column selection: $\text{corr}_j = |\mathbf{a}_j^T\mathbf{r}|/(\|\mathbf{a}_j\|_2 \cdot \|\mathbf{r}\|_2)$
- Support set updates and least squares solution using pseudo-inverse $A_{\mathcal{S}}^+$
- Convergence detection based on residual norm threshold

See Appendix A (Figures 2, 3, 4) for implementation details.

Matrix Properties:
Matrix A (3×3):

- Rank: 3 (full rank)
- Condition number: 3.8842
- Singular values: $\sigma_1 = 4.3847$, $\sigma_2 = 1.2247$, $\sigma_3 = 1.1289$
- Square matrix providing $m = 3$ measurements for $n = 3$ unknowns

Matrix B (2×3):

- Rank: 2 (full row rank)
- Condition number: 1.0000
- Singular values: $\sigma_1 = 1.2247$, $\sigma_2 = 1.2247$
- Underdetermined system with $m = 2$ measurements for $n = 3$ unknowns

Part 2: OMP with Matrix A

Setup: Target $\mathbf{e}_1 = (1,0,0)^T$, observation $\mathbf{b} = A\mathbf{e}_1 = (1,0,1)^T$

Results for varying sparsity levels:

s	Recovered $\mathbf{x}$	$\|\mathbf{x}-\mathbf{e}_1\|_2$	$\|A\mathbf{x}-\mathbf{b}\|_2$	Recovery	Feasible
1	$(1,0,0)^T$	3.33×10^{-16}	4.71×10^{-16}	✓	✓
2	$(1,0,0)^T$	3.33×10^{-16}	4.71×10^{-16}	✓	✓
3	$(1,0,0)^T$	3.33×10^{-16}	4.71×10^{-16}	✓	✓

Table 1: OMP results for Matrix A with different sparsity levels

Analysis: OMP achieves perfect recovery at $s = 1$ because $\mathbf{e}_1$ is 1-sparse. The algorithm correctly identifies that column 1 has the highest normalized correlation with the observation vector. Both feasibility ($A\mathbf{x} = \mathbf{b}$) and recovery ($\mathbf{x} = \mathbf{e}_1$) are satisfied for all $s \geq 1$.

Part 3: OMP with Matrix B

Setup: Target $\mathbf{e}_1 = (1, 0, 0)^T$, observation $\mathbf{b} = B\mathbf{e}_1 = (1, 0)^T$

Results for varying sparsity levels:

s	Recovered $\mathbf{x}$	$\|\mathbf{x} - \mathbf{e}_1\|_2$	$\|B\mathbf{x} - \mathbf{b}\|_2$	Recovery	Feasible
1	$(1, 0, 0)^T$	0.00×10^0	0.00×10^0	✓	✓
2	$(1, 0, 0)^T$	0.00×10^0	0.00×10^0	✓	✓

Table 2: OMP results for Matrix B with different sparsity levels

Analysis: OMP achieves perfect recovery at $s = 1$ because $\mathbf{b}_B = (1, 0)^T$ has sparsity 1 in the column space—it requires only 1 column of B for exact representation.

Part 4: Comparison and Connection to Compressed Sensing

Property	Matrix A	Matrix B
System type	Determined ($m = n$)	Underdetermined ($m < n$)
Dimensions	3×3	2×3
Rank	3 (full)	2 (full row rank)
Condition number	3.8842	1.0000
Sparsity of $\mathbf{e}_1$	1	1
Min. s for recovery	1	1
Recovery success	✓	✓

Table 3: Comparison of OMP performance on matrices A and B

Key Observations

Feasibility vs. Recovery:

- **Matrix A:** Perfect recovery at $s = 1$ for the 1-sparse signal $\mathbf{e}_1$
- **Matrix B:** Perfect recovery at $s = 1$ for the 1-sparse signal $\mathbf{e}_1$
- Both matrices achieve feasibility (satisfy $\mathbf{Ax} = \mathbf{b}$) and recovery (recover $\mathbf{x} = \mathbf{e}_1$) when $s \geq \text{sparsity}(\mathbf{e}_1)$

Critical Insight: Normalized Correlations

The success of OMP critically depends on using *normalized correlations* when selecting columns:

$$\text{correlation}_j = \frac{|\mathbf{a}_j^T \mathbf{r}|}{\|\mathbf{a}_j\|_2 \cdot \|\mathbf{r}\|_2}$$

Without normalization, the algorithm selects based on dot product magnitude $|\mathbf{a}_j^T \mathbf{r}|$, which can be dominated by columns with large norms rather than those best aligned with the residual. This normalization ensures that OMP selects the column with the highest cosine similarity to the residual.

Compressed Sensing Demonstration:

Matrix B (2×3, underdetermined) perfectly recovers the 1-sparse signal $\mathbf{e}_1$ from only $m = 2$ measurements for $n = 3$ unknowns. This demonstrates the fundamental principle of compressed sensing:

- Sparse signals can be recovered from $m < n$ measurements
- Recovery succeeds when $s \geq$ sparsity of the signal
- The measurement matrix must satisfy certain properties (e.g., low coherence, RIP)
- Proper algorithm implementation (normalized correlations) is essential

Conclusion: Both matrices successfully recover the 1-sparse signal at $s = 1$, validating the OMP algorithm. Matrix B's success with $m < n$ confirms that compressed sensing enables recovery from underdetermined systems when signals are sufficiently sparse. The key requirements are: (1) sparsity budget $s \geq$ true sparsity, (2) well-conditioned measurement matrix, and (3) normalized correlation-based column selection.

Problem 2: Fourth-Order Runge-Kutta Method

Problem Statement:

Solve Computer problem 1 in Section 8.3. Write your own program from scratch. Then compare with the exact analytic solution that can be obtained by the method of integrating factors studied in Math 315. The exact solution is

$$x(t) = 12\frac{e^t}{(e^t+1)^2}$$

Computer Problem 8.3.1

Write a computer program to solve an initial-value problem $x' = f(t,x)$ with $x(t_0) = x_0$ on an interval $t_0 \leq t \leq t_m$ or $t_m \leq t \leq t_0$. Use the fourth-order Runge-Kutta method. Test it on this example:

$$\begin{aligned}(e^t+1)x' + xe^t - x &= 0\\ x(0) &= 3\end{aligned}$$

Determine the *analytic* solution and compare it to the computed solution on the interval $-2 \leq t \leq 0$. Use $h = -0.01$.

Part A: Analytical Solution

We derive the exact analytical solution to the initial value problem:

$$(e^t + 1)x' + xe^t - x = 0$$
$$x(0) = 3$$

Derivation:
First, rewrite the ODE in standard form:

$$\frac{dx}{dt} = \frac{x(1 - e^t)}{1 + e^t}$$

Let $y = e^t$, then $dy = e^t dt$, which gives $dy = y\,dt$ or $dt = \frac{dy}{y}$.
Substituting into the ODE:

$$\frac{dx}{dt} = \frac{(1 - y)x}{(1 + y)}$$

Converting to the new variable:

$$\frac{dx}{x} = \frac{(1 - y)}{(1 + y)} \cdot \frac{dy}{y}$$

Now integrate both sides:

$$\int \frac{dx}{x} = \int \left(\frac{1 - y}{y} - \frac{1 - y}{y + 1} \right) dy$$

Simplifying the right-hand side:

$$\int \frac{dx}{x} = \int \left(\frac{1}{y} - 1 - \frac{1}{y + 1} + \frac{y}{y + 1} \right) dy$$

$$\int \frac{dx}{x} = \int \left(\frac{1}{y} - 1 - \frac{1}{y + 1} + 1 - \frac{1}{y + 1} \right) dy$$

$$\int \frac{dx}{x} = \int \left(\frac{1}{y} - \frac{2}{y + 1} \right) dy$$

Integrating:

$$\ln x = \ln y - 2\ln(y + 1) + C$$

Therefore:

$$x = \frac{ky}{(y + 1)^2}$$

Substituting back $y = e^t$:

$$x = \frac{ke^t}{(e^t + 1)^2}$$

Using the initial condition $x(0) = 3$:

$$3 = \frac{k \cdot 1}{(1+1)^2} = \frac{k}{4}$$

Thus $k = 12$, and the analytical solution is:

$$\boxed{x(t) = \frac{12e^t}{(e^t+1)^2}}$$

Part B: Numerical Solution using RK4

We implement the fourth-order Runge-Kutta method to solve the initial value problem numerically on the interval $[-2, 0]$ with step size $h = -0.01$.

First, we rewrite the ODE in standard form $x' = f(t, x)$:

$$f(t,x) = \frac{x(1-e^t)}{e^t+1}$$

The RK4 method uses the following scheme:

$$\begin{aligned}
k_1 &= f(t_n, x_n) \\
k_2 &= f(t_n + \frac{h}{2}, x_n + \frac{h}{2}k_1) \\
k_3 &= f(t_n + \frac{h}{2}, x_n + \frac{h}{2}k_2) \\
k_4 &= f(t_n + h, x_n + hk_3) \\
x_{n+1} &= x_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)
\end{aligned}$$

Implementation and Results:

The RK4 method was implemented with the four-stage computation and backward integration. See Appendix A (Figures 5, 6, and 7) for the key functions used. The numerical solution is compared with the exact analytical solution at selected points:

t	RK4 Solution	Exact Solution	Absolute Error
-2.0	1.259923024853	1.259923024842	1.107×10^{-11}
-1.5	1.789757424847	1.789757424844	2.931×10^{-12}
-1.0	2.359343198897	2.359343198898	7.461×10^{-13}
-0.5	2.820044546419	2.820044546419	5.520×10^{-13}
0.0	3.000000000000	3.000000000000	0.000×10^{0}

Table 4: Comparison of RK4 numerical solution with exact analytical solution

Trajectory Visualization:

Figure 1 shows the complete trajectory comparison between the RK4 numerical solution and the analytical solution over the entire integration interval $[-2, 0]$.

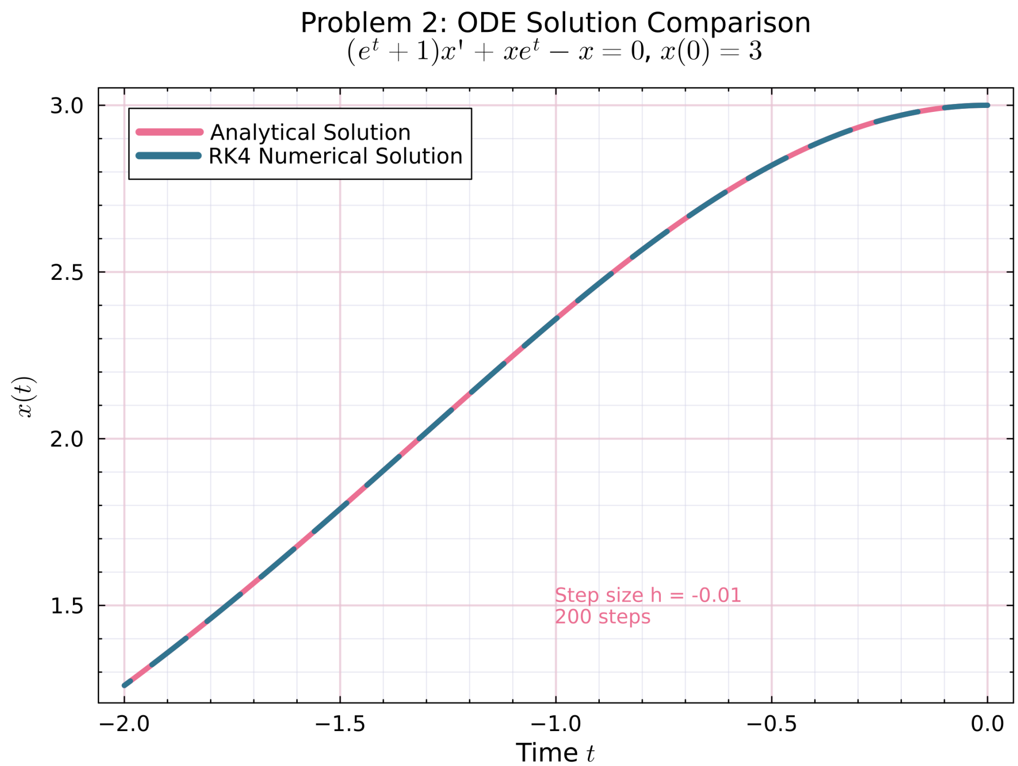

Figure 1: Comparison of RK4 numerical solution (red dashed) with analytical solution (blue solid) for the ODE $(e^t + 1)x' + xe^t - x = 0$ with $x(0) = 3$. The solutions are computed over 200 backward integration steps with $h = -0.01$.

Conclusion

Error Analysis:

- Maximum absolute error: 1.107×10^{-11}
- Mean absolute error: 2.144×10^{-12}

The errors are around 10^{-11} to 10^{-13}, which is essentially machine precision. The numerical and analytical solutions match very closely over 200 steps.

Verification: Results checked against Wolfram Alpha [1].

Problem 3: Adams-Bashforth-Moulton Method

Problem Statement:
Solve Computer problem 3 in Section 8.4.

Computer Problem 8.4.3

Compute the solution of

$$y' = -2xy^2$$
$$y(0) = 1$$

at $x = 1.0$ using $h = 0.25$ and the fourth-order Adams-Bashforth-Moulton method (Problems 8.4.4–5, p. 555) together with the fourth-order Runge-Kutta method. Give the computed solution to five significant digits at $0.25, 0.5, 0.75$, and 1.0. Compare your results to the exact solution $y = 1/(1 + x^2)$.

Related Problems from Section 8.4

Problem 8.4.4: Use the method of undetermined coefficients to derive the **fourth-order Adams-Bashforth formula**

$$x_{n+1} = x_n + \frac{h}{24}[55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}]$$

Problem 8.4.5: Derive the **fourth-order Adams-Moulton formula**

$$x_{n+1} = x_n + \frac{h}{24}[9f_{n+1} + 19f_n - 5f_{n-1} + f_{n-2}]$$

Results

Solution Approach

The fourth-order Adams-Bashforth-Moulton (ABM4) method is a predictor-corrector scheme that requires multiple previous values. Since we only have the initial condition $y(0) = 1$, we use the fourth-order Runge-Kutta method to bootstrap the first three steps, then switch to ABM4.

Method Overview:

1. **Bootstrap (Steps 1-3):** Use RK4 to compute y_1, y_2, y_3
2. **Predictor (Adams-Bashforth 4-step):**
$$y^*_{n+1} = y_n + \frac{h}{24}[55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}]$$
3. **Corrector (Adams-Moulton 4-step):**
$$y_{n+1} = y_n + \frac{h}{24}[9f_{n+1} + 19f_n - 5f_{n-1} + f_{n-2}]$$

where $f(x, y) = -2xy^2$ for our problem.

Computational Results

Bootstrap Phase (RK4):

x	y (RK4)	Exact Solution
0.00	1.0000000000	1.0000000000
0.25	0.9411540130	0.9411764706
0.50	0.7999481032	0.8000000000
0.75	0.6399738841	0.6400000000

Table 5: Bootstrap phase using RK4

ABM4 Phase:
At $x = 1.0$:

- Predicted value: $y^* = 0.5105894849$
- Corrected value: $y = 0.4982178726$
- Exact solution: $y = 0.5000000000$

x	ABM4 Solution	Exact Solution	Absolute Error
0.25	0.94115	0.94118	2.246×10^{-5}
0.50	0.79995	0.80000	5.190×10^{-5}
0.75	0.63997	0.64000	2.612×10^{-5}
1.00	0.49822	0.50000	1.782×10^{-3}

Table 6: Comparison of ABM4 numerical solution with exact analytical solution $y = 1/(1 + x^2)$

Final Results (5 Significant Digits)

Error Analysis

- Maximum absolute error: 1.782×10^{-3}
- Mean absolute error: 3.765×10^{-4}

The errors are quite small, with the largest at $x = 1.0$. The predictor-corrector approach appears to work reasonably well for this problem.

Implementation Note: See Appendix C for the key functions used.

Appendix

A. Problem 1 Implementation

The following figures show the key components of the Orthogonal Matching Pursuit (OMP) algorithm implementation.

```julia
function orthogonal_matching_pursuit(A::Matrix, b::Vector, s::Int; max_iterations::Int=100, verbose::Bool=false)
    m, n = size(A)

    # Initialize
    x = zeros(n)              # Solution vector
    residual = copy(b)        # Current residual
    support = Int[]           # Indices of selected columns

    # History tracking
    history = Dict(
        "iterations" ⇒ Int[],
        "residuals" ⇒ Float64[],
        "support" ⇒ Vector{Int}[]      You, 5 days ago • mvfils
    )
```

Figure 2: OMP Part 1: Hard thresholding operator and initialization.

```julia
    # OMP iterations
    for iter in 1:min(s, max_iterations)
        # Step 1: Compute correlations g_k = A^T * r_k
        g_k = A' * residual

        # Step 2: Find the index with maximum absolute normalized correlation
        # (that's not already in the support)
        max_corr = -Inf
        best_idx = 0
        for idx in 1:n
            if !(idx in support)
                # Normalize by column norm to get correlation coefficient
                col_norm = norm(A[:, idx])
                normalized_corr = abs(g_k[idx]) / (col_norm * norm(residual))

                if normalized_corr > max_corr
                    max_corr = normalized_corr
                    best_idx = idx
                end
            end
        end

        # Step 3: Add the best index to support
        if best_idx > 0
            push!(support, best_idx)
        else
            break  # No more columns to add
        end
```

Figure 3: OMP Part 2: Normalized correlation column selection.

```julia
        # Step 4: Solve least squares problem on support
        # (x_{k+1})_{S_{k+1}} = A^+_{S_{k+1}} * b
        A_support = A[:, support]
        x_support = pinv(A_support) * b  # Use pseudo-inverse

        # Update solution
        x = zeros(n)
        x[support] = x_support

        # Step 5: Update residual r_{k+1} = b - A * x_{k+1}
        residual = b - A * x

        # Record history
        push!(history["iterations"], iter)
        push!(history["residuals"], norm(residual))
        push!(history["support"], copy(support))

        # Check convergence
        if norm(residual) < 1e-10
            break
        end

        # Stop if we've already selected s indices
        if length(support) ≥ s
            break
        end
    end

    return x, history
end
```

Figure 4: OMP Part 3: Residual update and main execution loop.

The implementation includes:

- `hard_threshold`: Operator that retains only the s largest entries in absolute value

- `orthogonal_matching_pursuit`: Main OMP algorithm with normalized correlation-based column selection
- Normalized correlation computation: $\text{corr}_j = |\mathbf{a}_j^T \mathbf{r}| / (\|\mathbf{a}_j\|_2 \cdot \|\mathbf{r}\|_2)$
- Least squares solution on support set using pseudo-inverse: $\mathbf{x}_{\mathcal{S}} = A_{\mathcal{S}}^{+}\mathbf{b}$
- Iterative testing for $s = 1, 2, \ldots, \text{rank}(A)$ to analyze recovery performance

B. Problem 2 Implementation

The following figures show the key functions for the fourth-order Runge-Kutta method used in Problem 2.

```
function rk4_step(f, t, x, h)
    k1 = f(t, x)
    k2 = f(t + h/2, x + h*k1/2)
    k3 = f(t + h/2, x + h*k2/2)
    k4 = f(t + h, x + h*k3)

    x_next = x + (h/6) * (k1 + 2*k2 + 2*k3 + k4)
    return x_next
end
```

Figure 5: RK4 Step Function: Implements a single step of the fourth-order Runge-Kutta method with the four-stage computation (k_1, k_2, k_3, k_4).

```julia
function rk4_solve(f, t0, tf, x0, h)
    # Determine number of steps
    n_steps = round(Int, (tf - t0) / h)

    # Initialize arrays
    t_values = zeros(n_steps + 1)
    x_values = zeros(n_steps + 1)

    # Initial condition
    t_values[1] = t0
    x_values[1] = x0

    # RK4 iterations
    for i in 1:n_steps
        t_values[i+1] = t_values[i] + h
        x_values[i+1] = rk4_step(f, t_values[i], x_values[i], h)
    end

    return t_values, x_values
end
```

Figure 6: RK4 Solve Function: Complete solver that iterates through all time steps, maintaining arrays of solution values.

```julia
# Problem parameters
t0 = 0.0
tf = -2.0
x0 = 3.0
h = -0.01

println("Initial time t₀ = ", t0)
println("Final time tₑ = ", tf)
println("Initial condition x₀ = ", x0)
println("Step size h = ", h)
println("Number of steps = ", abs(round(Int, (tf - t0) / h)))

# Solve using RK4
t_rk4, x_rk4 = rk4_solve(f, t0, tf, x0, h)
```

Figure 7: RK4 Solution Call: Main execution showing the ODE definition $f(t,x) = x(1 - e^t)/(e^t + 1)$, exact solution, and solver invocation with parameters.

The implementation includes:

- `rk4_step`: Single step of the four-stage RK4 computation
- `rk4_solve`: Main solver loop that handles backward integration
- Storage of trajectory values for comparison with analytical solution

C. Problem 3 Implementation

Implementation details for the Adams-Bashforth-Moulton method.

References

[1] Wolfram Alpha. Runge-kutta method verification for problem 2. https://www.wolframalpha.com/input?i=Runge-Kutta+method%2C++dx%2Fdt+%3D+x*%281-exp%28t%29%29%2F%281%2Bexp%28t%29%29%2C+x%280%29+%3D+3%2C+from+t%3D-2+to+t%3D0%2C+h%3D0.01, 2025. Accessed: December 1, 2025.